

cluster for processing using the tool `SPARK_HOME/bin/spark-submit`.

A typical Spark application that is written in Scala looks like this:

```
import org.apache.spark.SparkContext
import org.apache.spark.SparkConf
import org.apache.spark.rdd.RDD
import org.apache.spark.rdd.RDD.toPairRDDFunctions

object HelloWorld {
  def main(args: Array[String]) {
    val sc = new SparkContext(new
    SparkConf().setAppName("HelloWorld"))
    print(sc.textFile("data.txt").
    flatMap(line => line.split(" ").count)
    sc.stop
  }
}
```

If this application is compiled and archived as `hello.jar`, it can be submitted to Spark with the following command:

```
SPARK_HOME/bin/spark-submit -class
HelloWorld -master <master-url> hello.
jar
```

Resilient distributed data set

Let us look at one of the jobs that we submitted to the Spark shell:

```
sc.parallelize(Array(1, 2, 3, 4, 5),
2).reduce((a,b)=>a+b)
```

As indicated earlier, this command is requesting the Spark context to partition the supplied data into two partitions. Each of the partitions is referred to as resilient distributed data set or RDD in short. The RDD is the fundamental abstraction in Spark. In other words, everything in Spark operates on RDDs behind the curtains.

The RDD is fault-tolerant and immutable. The workers work only on the allocated RDDs. Even if an executor crashes, another executor can be started with the same RDDs.

Primarily, there are two kinds of operations done on the RDDs — transformations and actions.

```
scala> :paste
// Entering paste mode (ctrl-D to finish)

val rdd = sc.parallelize(Array((1234, "Krishna"), (2345, "Mohan"), (3456, "Koyya")))
val df = spark.createDataFrame(rdd).toDF("eid", "name");
df.show

// Exiting paste mode, now interpreting.

+-----+-----+
| eid | name |
+-----+-----+
|1234|Krishna|
|2345| Mohan|
|3456| Koyya|
+-----+-----+

rdd: org.apache.spark.rdd.RDD[(Int, String)] = ParallelCollectionRDD[8] at parallelize at <paste>:24
df: org.apache.spark.sql.DataFrame = [eid: int, name: string]

scala> █
```

Figure 3: DataFrame

Transformations: A transformation accepts an RDD and maps it to a new RDD. Several transformations can form a pipeline. Transformations are lazy. In other words, Spark optimises the pipeline before actual execution. Operations like *map*, *filter*, *join*, etc, are some of the examples of transformations.

Actions: Actions return the results to the driver after computation. Operations like *reduce*, *collect* and *count* are some examples of actions.

Both transformations and actions are executed by loading the RDDs into the memory for best-in-class performance.

However, it is not recommended to program around just the RDDs, as there are better and high-level abstractions available on top of RDDs. Coding around RDDs is like writing assembly instructions when we have an option to use a high-level language.

Data sets and DataFrames

Spark holds data in the form of RDDs or in the form of DataFrames. A DataFrame is an abstraction built on top of an RDD, and is a distributed collection of structured data analogous to tables with schema.

DataFrames can be created from RDDs or directly from Spark Datasources like JDBC, Hive tables, etc. In other words, the DataFrames

can be created both from structured or semi-structured data.

Here is an illustration:

```
val rdd = sc.parallelize(Array((1234,
"Krishna"), (2345, "Mohan"), (3456,
"Koyya")))
val df = spark.createDataFrame(rdd).
toDF("eid", "name");
df.show
```

The above program indicates the context to create RDDs from the supplied array and then create a DataFrame from those RDDs. Figure 3 indicates that it is ok to say that DataFrames can be viewed as tables with schema.

Spark SQL

Once the DataFrame is created, it is quite easy to create a view of it and run operations that are similar to SQL operations.

Since we already have the DataFrame in the variable `df` in the above snippet, let's create a view named *employees* from the `df` with the following code:

```
df.createOrReplaceTempView("employees")
```

Now we are all set to execute SQL-like operations on this employee's view. Spark SQL supports the aggregations, restrictions and